A quantum decoder for quantum error correction

Using a quantum annealer to solve the decoding problem, and asking how it compares with the classical decoders the field runs today.

Yi-Zhuang You, CTO, EdenCode Inc · Yichen Hu, Assistant Professor of Physics, Florida Atlantic University · 9 September 2026 · draft v0.1

Summary

Every quantum computer that runs long enough to be useful needs quantum error correction, and at the heart of error correction is a good decoder: a program that, conditioned on the measured syndrome, infers which errors most likely occurred. Today's decoders run on classical hardware — matching algorithms, belief propagation, neural networks on CPUs and GPUs. The decoding problem itself, however, is a constrained binary minimization — exactly the class of problem a quantum annealer is built to attack.

We propose a quantum decoding algorithm that runs on a QPU instead, tested on the FAU D-Wave Advantage2 system and measured against standard classical decoders under matched conditions. If the annealer proves to be a strong solver of this problem, we further propose to use it as a **teacher**: its syndrome–error pairs become training data for a neural decoder that runs fast on a GPU. We treat both the benchmark and the distillation as open questions to be measured, not as expected outcomes.

THE QUESTION

Can one quantum machine help correct the errors of another? We want a measured answer — better, worse, or on par with classical decoders — on real annealing hardware.

Three phases

1. **Benchmark —** run the annealing decoder against matching, belief-propagation and tensor-network decoders on identical syndrome data, and measure the logical error rate and threshold.
2. **Distill —** train a neural decoder on annealer-generated syndrome–correction pairs, and test whether it keeps the accuracy while running fast enough to be used at runtime.
3. **Close the loop —** connect sampling, training and reinforcement learning into one pipeline, with the annealer as the data source.

Background: decoding is an optimization problem

A stabilizer code is described by a parity-check matrix H and a logical matrix L. Noise applies an unknown binary error vector e; what the hardware measures is only the syndrome $s = He$ over $\mathbb{F}_2$. The decoder infers the reverse: given s, propose the error most likely to have produced it.

$$\hat{e} \;=\; \arg\min_{e \in \{0,1\}^n} \; |e| \qquad \text{subject to} \quad He = s \pmod 2 \tag{Eq. (1)}$$

Eq. (1) is the minimum-weight formulation: among all error patterns consistent with what was measured, pick the lightest one. Log-likelihood weights make it noise-aware without changing its character. The problem is NP-hard in general, so every practical decoder is an approximation trading accuracy against speed.

Evaluation criteria

A correction does not have to recover the exact error to succeed — it has to leave the encoded information untouched. That condition is read off the logical matrix L: the correction $\hat{e}$ succeeds when the residual $\hat{e} \oplus e$ carries no logical operator, that is when $L(\hat{e} \oplus e) = 0$, and produces a **logical error** when it does not.

$$\mathrm{LER} \;=\; \Pr\big[\, L(\hat{e} \oplus e) \neq 0 \,\big] \tag{Eq. (2)}$$

The logical error rate of Eq. (2) is the figure of merit for every decoder, and the quantity we set out to measure. It also fixes the cost of the architecture: the logical error rate at a given physical error rate determines how many physical qubits a fault-tolerant machine must spend per logical qubit.

How it scales matters as much as its value. Below a critical physical error rate — the **error threshold** p_{th} — the logical error rate falls exponentially with the code distance d; above it, adding qubits makes things worse. The threshold depends on the code, the noise and the decoder together, so a better decoder raises it.

Mapping the decoder onto an annealer

A quantum annealer accepts one kind of input: a cost function that is quadratic in binary variables, which it then minimizes physically. Eq. (1) takes that form once the parity constraints move into the cost with a penalty weight — each check contributing a term that is zero when satisfied and positive when violated.

The cost function

$$E(e) = \sum_i w_i e_i + \lambda \sum_a \left(s_a - \bigoplus_{i \in a} e_i \right)^2 \qquad \text{Eq. (3)}$$

- `variables` — one binary variable per possible error location, so problem size tracks the number of physical qubits and measurement rounds.
- `weights` w — uniform for minimum-weight decoding, or log-likelihood weights for a noise-aware prior.
- `penalty` λ — large enough that violating a check never pays. Checks touching more than two error locations are not quadratic on their own and need auxiliary variables, so the encoding choice is part of the study.

Embedding Eq. (3) onto the Advantage2 connectivity graph, and choosing λ, the annealing schedule and the chain strength, is the practical work of Phase 1 — engineering choices we expect to matter as much as the formulation.

Why this is worth measuring

Every classical decoder buys speed with a modelling compromise. Matching decoders are fast and near-optimal on codes whose errors come in matchable pairs, but do not extend cleanly to general codes. Belief propagation with ordered-statistics post-processing handles general low-density codes, at a cost that grows with the depth of that post-processing. Tensor-network decoders come close to optimal but are slow enough to serve as a reference rather than a runtime tool.

An annealer represents a different compromise: it attacks the optimization directly rather than approximating it. Whether that yields better solutions on decoding instances is an open empirical question — these instances are sparse and highly degenerate, structurally unlike the dense problems annealers are usually benchmarked on. That is the reason to run them.

Research plan

Phase 1 · Benchmark the annealing decoder

We run the annealer as a decoder on standard code families and noise models, and compare it against classical decoders on identical syndrome data. Preliminary work on the surface code indicates the formulation runs and returns valid corrections; what we have not yet measured is the quantity that matters — the logical error rate of Eq. (2), and the error threshold that follows from it.

CODES & NOISE

- Unrotated surface code, several distances
- Color code
- Bivariate bicycle and related qLDPC codes
- Code-capacity and phenomenological noise

BASELINES

- Minimum-weight perfect matching
- Belief propagation with OSD post-processing
- Tensor network, as a near-optimal reference
- Simulated annealing and tabu on the same cost function

The classical-solver baselines matter as much as the decoder baselines: running simulated annealing on Eq. (3) separates what the formulation contributes from what the quantum hardware contributes.

PHASE 1 DELIVERABLE

Logical error rate against physical error rate for each code, decoder and distance, on shared syndrome sets — and the threshold estimate that follows from those curves.

Phase 2 · Distill a fast decoder from annealer data

Suppose Phase 1 finds the annealer solves these instances better than the classical decoders. It would still not be a practical runtime decoder: annealing access is expensive, and its latency is far above the microsecond budget a superconducting processor allows. A hypothesis worth testing is that its value is as a **teacher** rather than as a runtime component.

Research plan, continued

Each annealer call produces a syndrome–correction pair. Collected at scale, these pairs form a supervised training set for a neural decoder that runs on a GPU. The question we would ask in Phase 2 is whether a student network trained on annealer solutions inherits their accuracy while running orders of magnitude faster. EdenCode's existing neural decoders reach sub-millisecond inference per shot on GPU, a regime already relevant to neutral-atom and trapped-ion machines, where cycle times are long.

Phase 3 · Close the loop

The final phase connects the pieces into one pipeline: sample decoding problems on the annealer, train the neural decoder on what comes back, then let the trained decoder improve on its teacher through reinforcement learning against known outcomes. The analogy is language-model training: imitation of a strong teacher establishes a competent base model, and reinforcement on real outcomes takes it further.

Broader impact and deliverables

Quantum annealing is often asked which commercially meaningful problem it solves. This project proposes one answer from inside the quantum industry itself: helping another quantum computer stay coherent. A demonstration that one quantum machine can supply the decoding intelligence another needs is a concrete application story, whichever way the benchmark turns out.

- Open benchmark suite — decoding instances, embeddings and annealing parameters, released so results are reproducible on the same machine.
- Measured comparison of annealing and classical decoders on logical error rate and threshold.
- A published dataset of annealer-generated syndrome–correction pairs for the machine-learning community.
- Student training at Florida Atlantic University on error correction and annealing hardware, and at least one joint publication.

References and scope

Machine time, data volumes and staffing are set out in the accompanying allocation request; the science questions in this document are what that time would buy.

[1] Dennis, E., Kitaev, A., Landahl, A., Preskill, J. — Topological quantum memory. J. Math. Phys. 43, 4452, 2002. arXiv:quant-ph/0110143

[2] Panteleev, P., Kalachev, G. — Degenerate quantum LDPC codes with good finite length performance. Quantum 5, 585, 2021. arXiv:1904.02703

[3] Bravyi, S. et al. — High-threshold and low-overhead fault-tolerant quantum memory. Nature 627, 778, 2024. arXiv:2308.07915

[4] Higgott, O., Gidney, C. — Sparse Blossom: correcting a million errors per core second with minimum-weight matching. Quantum 9, 1600, 2025. arXiv:2303.15933

[5] Bausch, J. et al. — Learning high-accuracy error decoding for quantum processors. Nature 635, 834, 2024. arXiv:2310.05900

[6] Chancellor, N., Zohren, S., Warburton, P. — Circuit design for multi-body interactions in superconducting quantum annealing systems with applications to a scalable architecture. npj Quantum Inf. 3, 21, 2017. arXiv:1603.09521